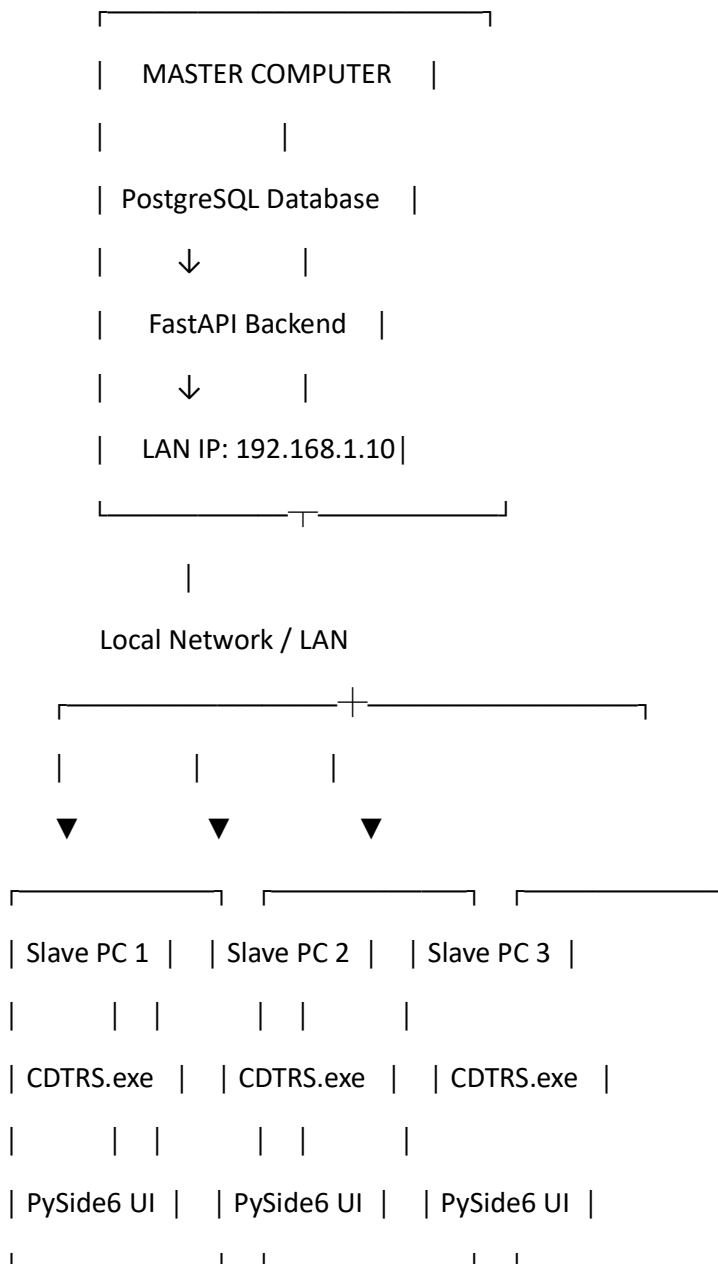


Yes. For your **CDTRS PySide6 + Python + PostgreSQL application**, this is actually a very normal architecture for an internal/local network deployment.

If you have:

- **1 Master computer** → backend + PostgreSQL
- **Multiple Slave/client computers** → only PySide6 .exe
- All computers connected to the **same LAN/Wi-Fi**

then you can deploy it like this:



The important point

The frontend EXE should NOT directly connect to PostgreSQL.

Instead:

PySide6 EXE



HTTP/HTTPS API



FastAPI backend



PostgreSQL

This is much better for your CDTRS architecture because your backend already handles things such as authentication, routing, document state, permissions, Outlook synchronization, OCR, etc.

1. What goes on the Master PC?

Your master computer would have three things:

MASTER PC

CDTRS Backend



├— FastAPI

├— SQLAlchemy

├— OCR services

├— APScheduler

├— Outlook integration

└— other backend modules

PostgreSQL

```
|
└─ CDTRS database
```

Backend configuration

```
|
└─ .env
└─ database credentials
```

For example:

```
C:\CDTRS\
|
└─ backend\
|   └─ main.py
|   └─ app\
|   └─ models\
|   └─ routers\
|   └─ services\
|   └─ requirements.txt
|   └─ .env
|
└─ scripts\
    └─ start_backend.bat
    └─ backup_database.bat
```

PostgreSQL can run as a **Windows Service**, so you don't need to manually open PostgreSQL every time.

FastAPI can run as a service as well.

2. Give the Master PC a fixed LAN IP

This is important.

Suppose the master computer gets:

192.168.1.100

Your backend might run on:

192.168.1.100:8000

So from another computer:

http://192.168.1.100:8000

would reach your backend.

For example:

Slave PC

CDTRS.exe



http://192.168.1.100:8000



Master PC



FastAPI



PostgreSQL

Ideally, configure the router/DHCP reservation so the master keeps the same IP.

Even better, you can use a local hostname such as:

http://cdtrs-server:8000

instead of hardcoding an IP, if your network/DNS setup supports it.

3. What goes on the Slave PCs?

Very little.

You create:

CDTRS.exe

using something like PyInstaller.

The slave computer gets:

CDTRS.exe

and perhaps:

config.json

containing:

```
{  
  "backend_url": "http://192.168.1.100:8000"  
}
```

Then:

User double-clicks CDTRS.exe



PySide6 application



Reads backend URL



HTTP request to Master PC



FastAPI backend



PostgreSQL

The slave PC **does not need PostgreSQL installed**.

It also doesn't need your backend Python environment.

It only needs whatever dependencies are bundled into the EXE.

4. Your PySide6 code should have a configurable backend URL

Don't do this everywhere:

```
requests.get("http://localhost:8000/documents")
```

Because on Slave PC:

localhost

means **the Slave PC itself**, not your Master PC.

Instead:

```
BACKEND_URL = "http://192.168.1.100:8000"
```

Then:

```
requests.get(  
    f"{BACKEND_URL}/documents"  
)
```

Even better, put it in a configuration file.

For example:

config.json

```
{  
    "backend_url": "http://192.168.1.100:8000"  
}
```

Your EXE reads it:

```
import json
```

```
with open("config.json", "r") as f:
```

```
    config = json.load(f)
```

```
BACKEND_URL = config["backend_url"]
```

That way, if your backend moves from:

192.168.1.100

to:

192.168.1.105

you don't have to rebuild the EXE.

5. How the backend communicates with PostgreSQL

On the Master PC:

FastAPI



SQLAlchemy



PostgreSQL

For example your .env could contain:

```
DATABASE_URL=postgresql://cdtrs_user:password@localhost:5432/cdtrs
```

Notice:

localhost

is perfectly fine **inside the backend**, because PostgreSQL is on the same Master computer.

The important distinction is:

Slave → Backend

192.168.1.100:8000

Backend → Database

localhost:5432

So you don't expose PostgreSQL to all client machines.

6. You should NOT expose PostgreSQL to the slave computers

This is an important architectural decision.

Don't make this:

Slave 1 ———┐

Slave 2 ———┐

Slave 3 ———┐——→ PostgreSQL

Slave 4 ———┐

Instead:

Slave 1 —→

Slave 2 —→

Slave 3 —→ FastAPI —→ PostgreSQL

Slave 4 —→

This gives you:

- centralized authentication
- centralized authorization
- centralized validation
- easier database management
- no database credentials on client PCs
- easier updates
- easier auditing
- better security

And this fits your CDTRS design particularly well because permissions depend on roles like:

Administrator

Director Secretary

Director

HOD

Employee

TSO

The backend should decide what each user is allowed to do.

7. How do you actually run the backend?

You have several options.

Option A — Start manually

On Master:

```
cd C:\CDTRS\backend
```

```
venv\Scripts\activate
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The important part is:

--host 0.0.0.0

NOT:

--host 127.0.0.1

Because:

127.0.0.1

means only the Master can access it.

Whereas:

0.0.0.0

means listen on the machine's network interfaces.

8. Better: make FastAPI start automatically

For an actual deployment, I would recommend making the backend a Windows service or using a service manager.

Then the Master PC boots:

Windows starts

↓

PostgreSQL service starts

↓

CDTRS Backend service starts

↓

Backend listens on port 8000

↓

Users can start CDTRS.exe

Nobody has to manually run Python.

9. Windows Firewall is important

Suppose the Master is:

192.168.1.100

and FastAPI:

port 8000

Windows Firewall may block incoming connections.

You therefore need a firewall rule allowing inbound TCP:

Port: 8000

from your internal LAN.

Then Slave PC can access:

http://192.168.1.100:8000

You should **not blindly open the port to public/internet networks**.

For an internal CDTRS deployment, restrict it to the organization's private network.

10. What about PostgreSQL?

Install PostgreSQL **only on the Master**.

For example:

Master PC

PostgreSQL

↓

Database: cdtrs

↓

Tables

├— users

├— documents

├— document_branches

├— departments

└─ notifications

└─ audit_logs

└─ ...

You can keep PostgreSQL listening only locally if FastAPI is on the same machine:

localhost:5432

That is actually preferable.

You don't need:

Slave → PostgreSQL

at all.

11. What happens when a user uploads a document?

For example, Employee PC:

Employee

↓

CDTRS.exe

↓

Select PDF

↓

POST /documents

↓

FastAPI on Master

↓

Store document

↓

PostgreSQL stores metadata

↓

Backend performs routing/workflow

↓

Response to PySide6



UI updates

The actual PDF could be stored on the Master server filesystem, for example:

C:\CDTRS\data\documents\

while PostgreSQL stores metadata:

document_id

filename

department

uploaded_by

status

priority

created_at

...

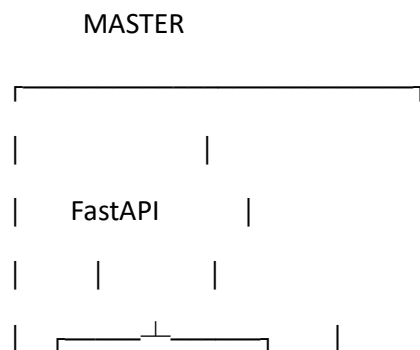
Or you can use another centralized storage solution.

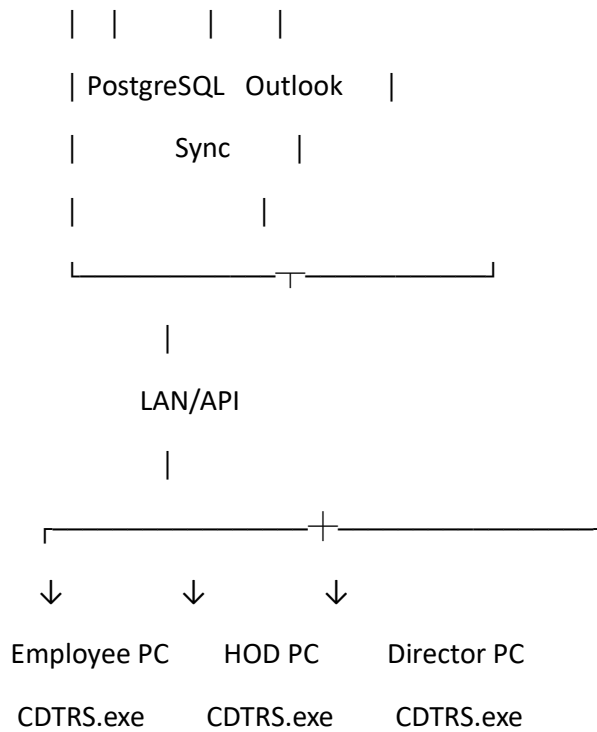
12. This is especially important for your Outlook integration

Your CDTRS architecture has Outlook synchronization.

If Outlook is associated with the organization's central account/mailbox, you probably want the **backend/service on the Master** to handle synchronization rather than every slave EXE doing it independently.

Conceptually:





That prevents multiple clients from independently processing the same incoming Outlook correspondence.

13. Your OCR can also remain on the Master

Since your project uses things such as:

- PaddleOCR
- TrOCR
- PyMuPDF
- rapidfuzz
- spaCy/regex
- routing logic

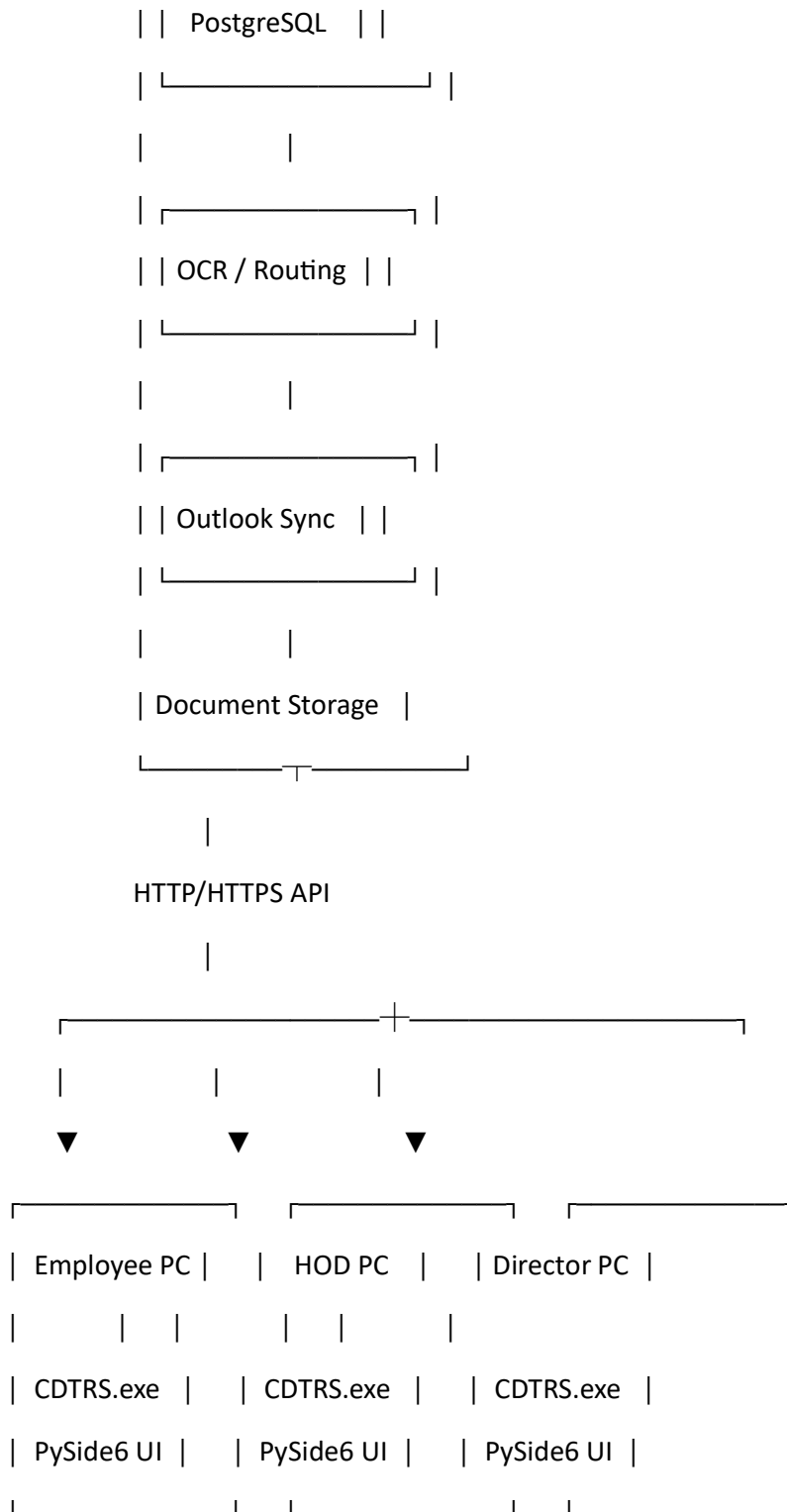
you don't necessarily want every client PC carrying all those dependencies.

Instead:

Client



Upload document



15. One thing I would change in your terminology

Rather than calling them:

Master computer and Slave computers

for your actual project documentation, I'd recommend:

Server / Client architecture

or:

Central Server + Client Workstations

So your documentation can say:

CDTRS follows a centralized client-server architecture. The central server hosts the FastAPI backend, PostgreSQL database, document storage, OCR services and background synchronization processes. Client workstations run a lightweight PySide6 desktop application that communicates with the backend through REST APIs over the organization's local network.

That sounds much more professional for a project report/viva.

16. What the final installation could look like

Server

CDTRS Server

|

|— PostgreSQL

|

|— CDTRS Backend

| |— FastAPI

| |— SQLAlchemy

| |— OCR

| |— Routing

| |— Outlook Sync

| |— Scheduler

|

|— Data

| |— Documents

| └─ Attachments

| └─ Logs

|

└─ Backup

Client

CDTRS Client

|

└─ CDTRS.exe

└─ config.json

That's it.

17. And yes — you can distribute only the EXE

You don't need to give every computer:

Python

PostgreSQL

FastAPI

SQLAlchemy

PaddleOCR

PyMuPDF

etc.

The client machine can simply have:

CDTRS/

|

└─ CDTRS.exe

└─ config.json

The EXE contains your PySide6 application and its client-side Python dependencies.

Then:

CDTRS.exe



192.168.1.100:8000



FastAPI



PostgreSQL

That's the cleanest approach.

One important caveat: if your PySide6 frontend currently performs database operations directly with SQLAlchemy/PostgreSQL, you'll need to refactor those calls so the frontend talks to your FastAPI API instead. If you've already separated your frontend/backend properly, you're basically ready for this architecture.